

BIMM 143 Lab 6 Hw

Mayla (PID: A19136116)

2026-04-19

Section 1.A.

```
normalize <- function(x) {  
  (x - min(x, na.rm = TRUE)) / (max(x, na.rm = TRUE) - min(x, na.rm = TRUE))  
}
```

```
df <- data.frame(a=1:10,  
                 b=seq(200, 400, length=10),  
                 c=11:20,  
                 d = rep(NA, 10))
```

```
df$a <- normalize(df$a)  
df$b <- normalize(df$b)  
df$c <- normalize(df$c)  
df$d <- normalize(df$d)
```

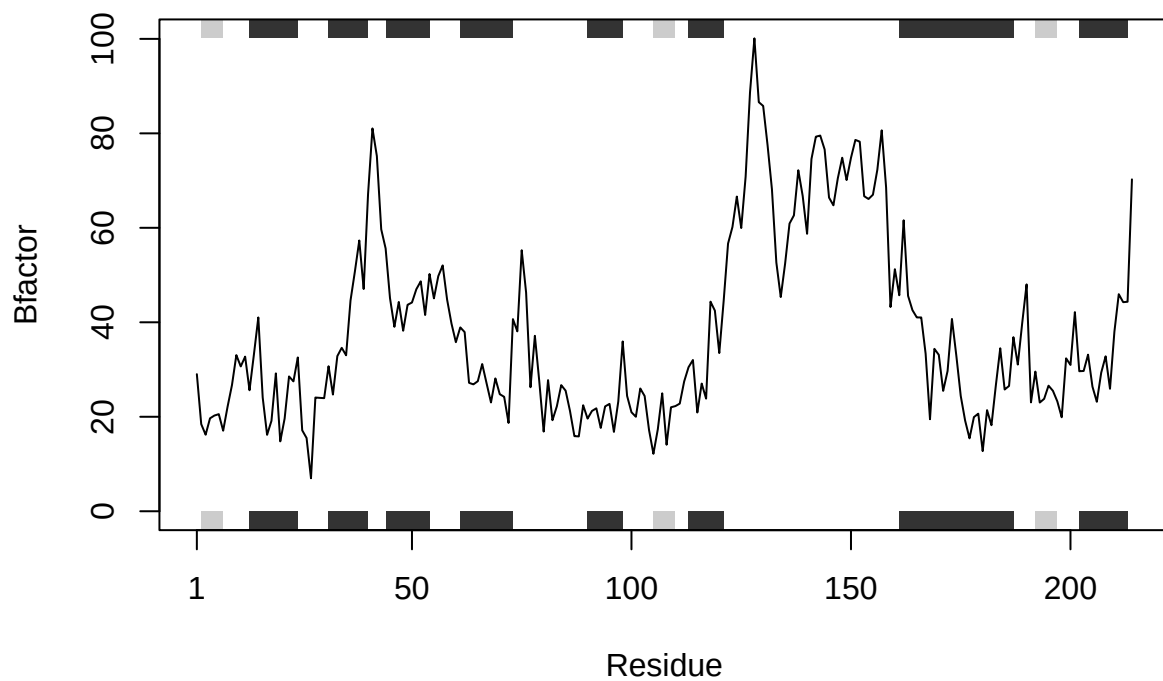
```
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min; returning Inf  
## Warning in max(x, na.rm = TRUE): no non-missing arguments to max; returning  
## -Inf  
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min; returning Inf
```

Section 1.B.

```
# New and Improved code  
library(bio3d)  
analyze_pdb <- function(pdb_id) {  
  pdb <- read.pdb(pdb_id)  
  chainA_ca <- trim.pdb(pdb, chain = "A", eley = "CA")  
  b_factors <- chainA_ca$atom$b  
  plotb3(b_factors, sse = chainA_ca, typ = "l", ylab = "Bfactor")  
  return(b_factors)  
}
```

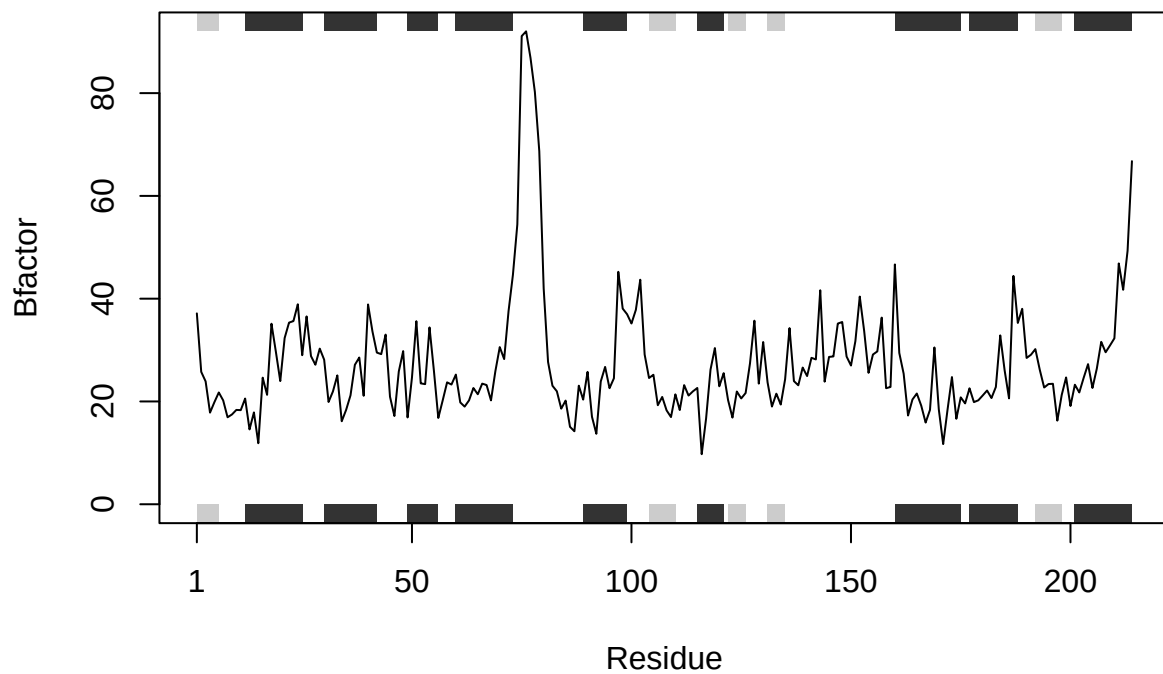
```
s1.b <- analyze_pdb("4AKE")
```

```
## Note: Accessing on-line PDB file
```



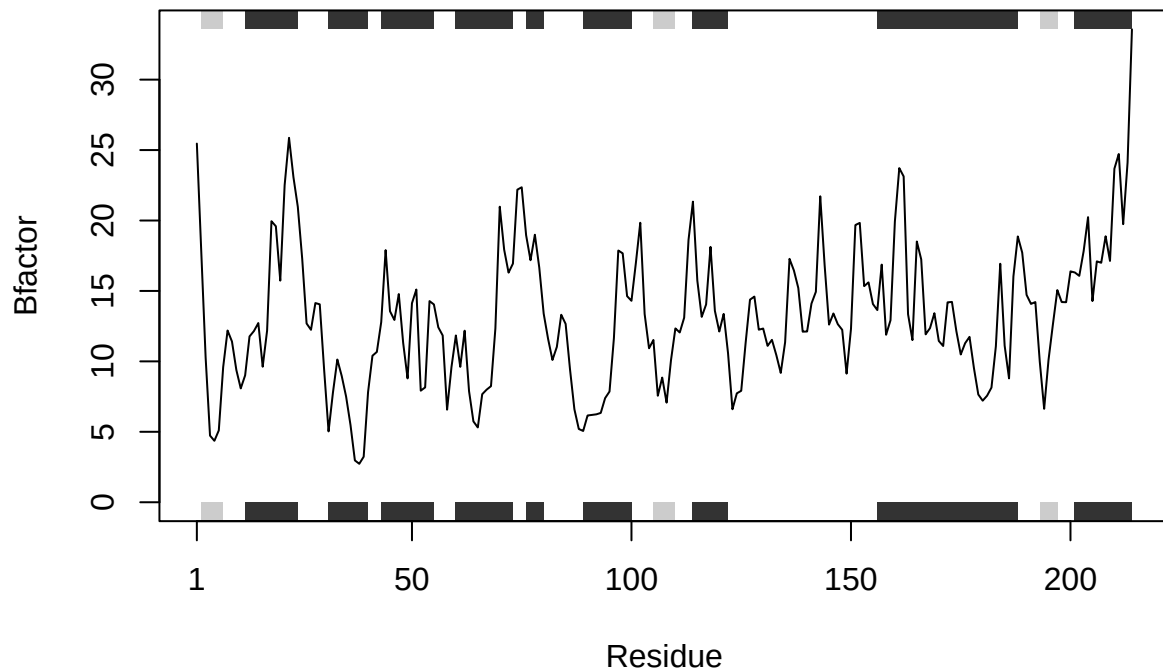
```
s2.b <- analyze_pdb("1AKE")
```

```
## Note: Accessing on-line PDB file  
## PDB has ALT records, taking A only, rm.alt=TRUE
```



```
s3.b <- analyze_pdb("1E4Y")
```

```
## Note: Accessing on-line PDB file
```



Q1. What does your function do? This function reads a PDB file, extracts the alpha carbon atoms from chain A, calculates the B-factors, and plots them to visualize protein flexibility.

Q2. What does the `trim.pdb()` function do? `trim.pdb()` subsets a PDB structure. In this case, it keeps only chain A and only alpha-carbon (CA) atoms, so the analysis focuses on one comparable backbone trace per protein.

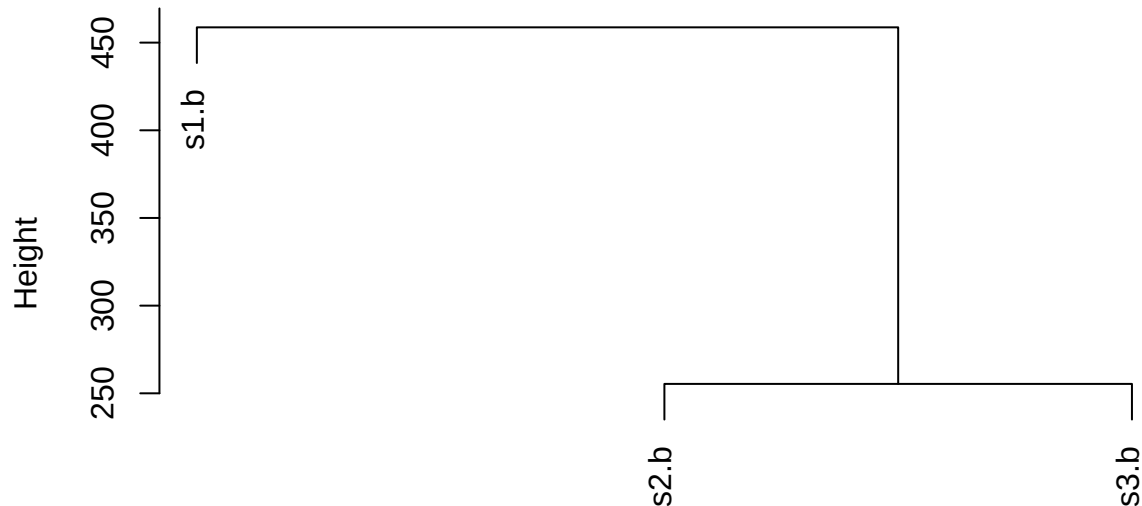
Q3. What input parameter would turn off the marginal black and grey rectangles in the plots and what do they represent in this case? Set the `sse = NULL` argument in `plotb3()` to turn off the marginal black and grey rectangles. In these plots, those rectangles represent secondary structure elements, mainly alpha helices and beta strands/sheets, mapped along the protein sequence.

Q4. What would be a better plot to compare across the different proteins? A better plot would be a single overlaid line plot with all three B-factor traces on the same axes, or a faceted plot with the same y-axis scale for each protein. That makes direct comparison much easier than looking at three separate plots.

Q5. Which proteins are more similar to each other in their B-factor trends. How could you quantify this? **HINT:** try the `rbind()`, `dist()` and `hclust()` functions together with a resulting dendrogram plot. Look up the documentation to see what each of these functions does:

```
hc <- hclust( dist( rbind(s1.b, s2.b, s3.b) ) )
plot(hc)
```

Cluster Dendrogram



```
dist(rbind(s1.b, s2.b, s3.b))
hclust (*, "complete")
```

The proteins that cluster closest together in the dendrogram have the most similar B-factor trends. To quantify similarity, I can combine the B-factor vectors with `rbind()`, calculate pairwise distances with `dist()`, and cluster them with `hclust()`. Smaller distances indicate more similar B-factor profiles.

Q6. How would you generalize the original code above to work with any set of input protein structures? I would generalize the code by writing a function that takes any PDB ID as input, reads the structure, trims it to the desired chain/atom type, extracts B-factors, and plots the result. This removes repetition and lets the same analysis work on any protein structure by changing only the function input.

Function Description

This function takes a PDB ID as input, reads the protein structure, extracts chain A alpha carbons, and plots B-factor values.

Inputs

- `pdb_id`: a string representing the PDB ID (e.g. "4AKE")

Output

- A plot of B-factor values
- Returns the B-factor vector